METHODOLOGY & MATHEMATICS

Scheduling at the Limit

How a full year of resident call was modeled, optimized, and verified — and what made the problem hard.

10^644	6,363	8,400	0
POSSIBLE SCHEDULES, UNCONSTRAINED	DECISION VARIABLES	CONSTRAINTS	RULE CONFLICTS REMAINING

In brief. The 2026–2027 call calendar assigns one intern and one senior resident to each of **354** nights. Rebuilding it fairly is not a clerical task but a large combinatorial optimization problem. This note walks through the modeling, the mathematics of "fairness," the solver that cracked it, and the statistical fingerprints the process left behind — including how ten of eleven senior residents ended the year carrying an *identical* weighted on-call burden.

Prepared for the residency leadership · Constraint-programming model (Google OR-Tools CP-SAT) · Python

§ 1 A small calendar, an enormous space

Seventeen residents — six interns, seven second-years, four third-years — must cover every night of the academic year. Each call day needs exactly one intern and one senior; most days that is the whole story, and the year runs to 354 scheduled days. It looks like a grid you could fill in by hand, and indeed it usually is. But the moment you ask for the *fairest* filling that still obeys every rule, the grid turns into one of the largest objects most people will ever reason about.

Count the raw possibilities. On a given night there are 6 interns and 11 seniors who *could* be chosen, so $6 \times 11 = 66$ pairings. Across 354 independent nights that is 66^{354} — a number with **645 digits**. Written out, it begins 1.3 and trails off into six hundred more zeros than there are atoms in the observable universe.

FASCINATING FACT № 1 — COSMIC SCALE

The observable universe holds roughly 10^{80} atoms. The unconstrained space of call schedules is about 10^{644}. The schedule problem is therefore larger than the universe by a factor of 10^{564} — you could give every atom its own universe of atoms, more than seven times over, and still not catch up.

No computer enumerates a space like that; the heat death of the sun arrives first. The escape is that the rules which make the calendar *look* harder are exactly what make it *solvable*. Every constraint — a rotation a resident cannot leave, a Sunday a class never works — slices the space down. The art is to describe those slices precisely enough that a solver can navigate what remains by logic rather than brute force.

§ 2 From spreadsheet to a model the math can see

Two workbooks came in: the draft call calendar (twelve month tabs) and the rotation master schedule. The first job was to turn that human-readable grid into a structured model — and to *prove* the model was faithful before trusting a single optimization on top of it.

The faithfulness test was simple and strict. The workbook keeps its own running tallies on a year-to-date tab. When the reconstructed model was tallied the same way, it reproduced the tab to the shift: **706 total assignments**, every resident's count matching exactly. Only then did the rotation grid get compiled into the language of constraints — producing **1,143** forbidden (resident, day) pairs, each one a night a particular resident is on a rotation that the rules bar from call.

706	1,143	22
SHIFTS RECONSTRUCTED — MATCHED THE YTD TAB EXACTLY	FORBIDDEN RESIDENT-DAY PAIRS FROM ROTATION RULES	RULE & PTO CONFLICTS FOUND IN THE DRAFT

The draft was faithful but not clean: 19 rotation conflicts, 3 PTO/instruction conflicts, plus back-to-back calls and winter-holiday coverage gaps surfaced once the rules were machine-checked.

§ 3 Two answers: repair, or rebuild

Two deliverables follow from one diagnosis. They answer different questions, and which one you adopt depends on appetite for change.

VERSION 1

Minimal Repair

- Fixes **only** the rule violations — nothing else moves.
- Each fix is a swap between two residents on the **same day of the week**.
- Every resident keeps their exact totals and day-mix; only the conflicting dates change.
- Smallest possible disruption to your draft.

VERSION 2

Full Rebuild

- Re-derives the **entire year** from the rules up.
- Optimized so the on-call burden is as level as the rules allow.
- R3s off Sundays; R2s at 7–8 Sundays; winter holidays covered by interns.
- Patel's extra early-year backup days deliberately preserved.

FASCINATING FACT № 2 — WHY THE REPAIR IS "INVISIBLE" ON THE TALLY TABS

Version 1 only ever swaps two residents who are both on call on, say, a *Friday*. Because the swap trades one Friday for another, every resident's count of Mondays, Tuesdays, … Sundays is left untouched — the swap is a *permutation that preserves each marginal distribution*. That is why the workbook's year-to-date totals are numerically identical before and after the repair: 20 conflicts vanish without moving a single tally.

§ 4 The model: variables, constraints, and one honest objective

The rebuild is a **constraint-optimization problem**. Every choice is a yes/no switch; every rule is an equation those switches must satisfy; and a single objective function decides which of the astronomically many legal schedules is best.

THE SWITCHES. For each resident *r* and day *d* a binary variable $x_{r,d}$ is 1 if *r* is on call that day. That is 2,124 intern switches and 3,894 senior switches — **6,018** primary decisions, joined by auxiliary variables for a model total of **6,363**.

MODEL.PY — DECISION VARIABLES & COVERAGE — PYTHON · OR-TOOLS

```
for s in SEN:
    for d in days:
        xs[s,d] = model.NewBoolVar(f"s_{s}_{d}")   # 1 ⇔ senior s on call day d

for d in days:
    model.Add(sum(xi[i,d] for i in INT) == 1)              # exactly one intern
    model.Add(sum(xs[s,d] for s in SEN) == 1 + extra[d])  # one senior (2 on backup days)
```

THE RULES. Eight thousand four hundred constraints, but only a handful of *kinds*. They sort cleanly into a taxonomy:

CONSTRAINT FAMILY	WHAT IT ENFORCES	COUNT
Coverage	One intern + one senior every night (two seniors on backup days)	708
Eligibility	No call while on a no-call or day-restricted rotation	1,143
Night float	First-half interns fixed to their night-float weeknights	120
Post-call rest	No resident on call two days running	353×
Structure	R3 no Sunday; R2 7–8 Sundays & 14–16 weekend nights	·
Holiday equity	≤ one major / two minor per senior; the four "mix-up" holidays on four interns	·
Fairness bands	Per-class caps on Saturdays, Fridays, Sundays	·
Total constraints in the rebuilt model		8,400

§ 5 What "fair" means in equations

Fairness has to be made numeric before a computer can pursue it. Not every call night is equally unwelcome, so each day of the week carries an *inconvenience weight*: a quiet Wednesday counts 1; a Saturday, the worst, counts 4.

$$w: \quad \textit{Wed, Thu} = \mathbf{1} \;\cdot\; \textit{Mon, Tue} = \mathbf{2} \;\cdot\; \textit{Fri, Sun} = \mathbf{3} \;\cdot\; \textit{Sat} = \mathbf{4}$$

Day-of-week inconvenience weights

A resident's **weighted load** is then the sum of those weights over every night they work — a single number capturing not just *how much* call they take but *how painful* it falls:

$$W_r = \sum_{d \in D} w_{dow(d)} \cdot x_{r,d}$$

Weighted on-call load for resident r

The objective is then to make those loads as *equal* as the rules permit. Within every class the solver minimizes the gap between the busiest and least-busy resident — a classic `min–max` fairness criterion — summed across classes and added to the day-by-day balancing terms:

$$\text{minimize} \quad \sum_{\text{classes } c} \left(\max_{r \in c} W_r - \min_{r \in c} W_r \right) + \textit{(day-type spread)}$$

A flatter distribution scores lower; perfect equality scores zero

FASCINATING FACT № 3 — A SIMPSON'S-PARADOX TRAP

The first rebuild balanced each class *internally* and looked perfect: every R2 identical, every R3 identical. Yet it was grossly unfair *between* classes — the solver had quietly handed all the gentle Wednesdays to the R2s (average inconvenience **2.09**) and all the brutal Mondays and Fridays to the R3s (**2.55**). Fairness within every group did not add up to fairness overall. The cure was to add *cross-cohort* balancing terms, after which the two classes converged to the same load. It is the scheduling twin of Simpson's paradox: a trend true in every subgroup can reverse for the whole.

§ 6 Solving it — logic, not luck

The model was handed to **CP-SAT**, Google OR-Tools' constraint solver. Despite the name, it does not guess-and-check; it translates the problem into Boolean satisfiability and reasons with *lazy clause generation* — learning a new logical fact from every dead end so that whole regions of the 10^{644} space are eliminated at a stroke.

MODEL.PY — THE PROGRAM'S STRUCTURAL RULES — PYTHON · OR-TOOLS

```python
# R3 residents take no Sunday call (a single Lux exception is permitted)
model.Add(sum(xs[s,d] for s in R3 for d in SUN) <= 1)

for s in R2:                                  # every R2 carries a weekend share
    model.Add(sum(xs[s,d] for d in SUN)      >= 7)
    model.Add(sum(xs[s,d] for d in WKND) <= 16)
```

Eight parallel search workers ran under a 240-second budget. On the core formulation the solver did something brute force never can: it returned a certificate of **global optimality** — a proof that no legal schedule scores better, not merely that none was found. Reaching the final calendar took three deliberate iterations: the degenerate split of §5, then a smoothing pass so the one intentionally heavier resident did not absorb a lopsided pile of Mondays.

> **Ten of the eleven seniors finished the year carrying a weighted load of exactly 75 — the same number, to the unit.**

§ 7 One resident kept deliberately different

Not every difference is unfairness. The draft had one second-year, Patel, carrying twelve extra "second-senior" backup days early in the year — a real feature of an off-cycle schedule, and one to *keep*.

The rebuild preserves those twelve days and his elevated total. Ten of the original dates survive untouched; the two that were themselves rule violations — a Nexplanon-training "do-not-schedule" night and a Sunday

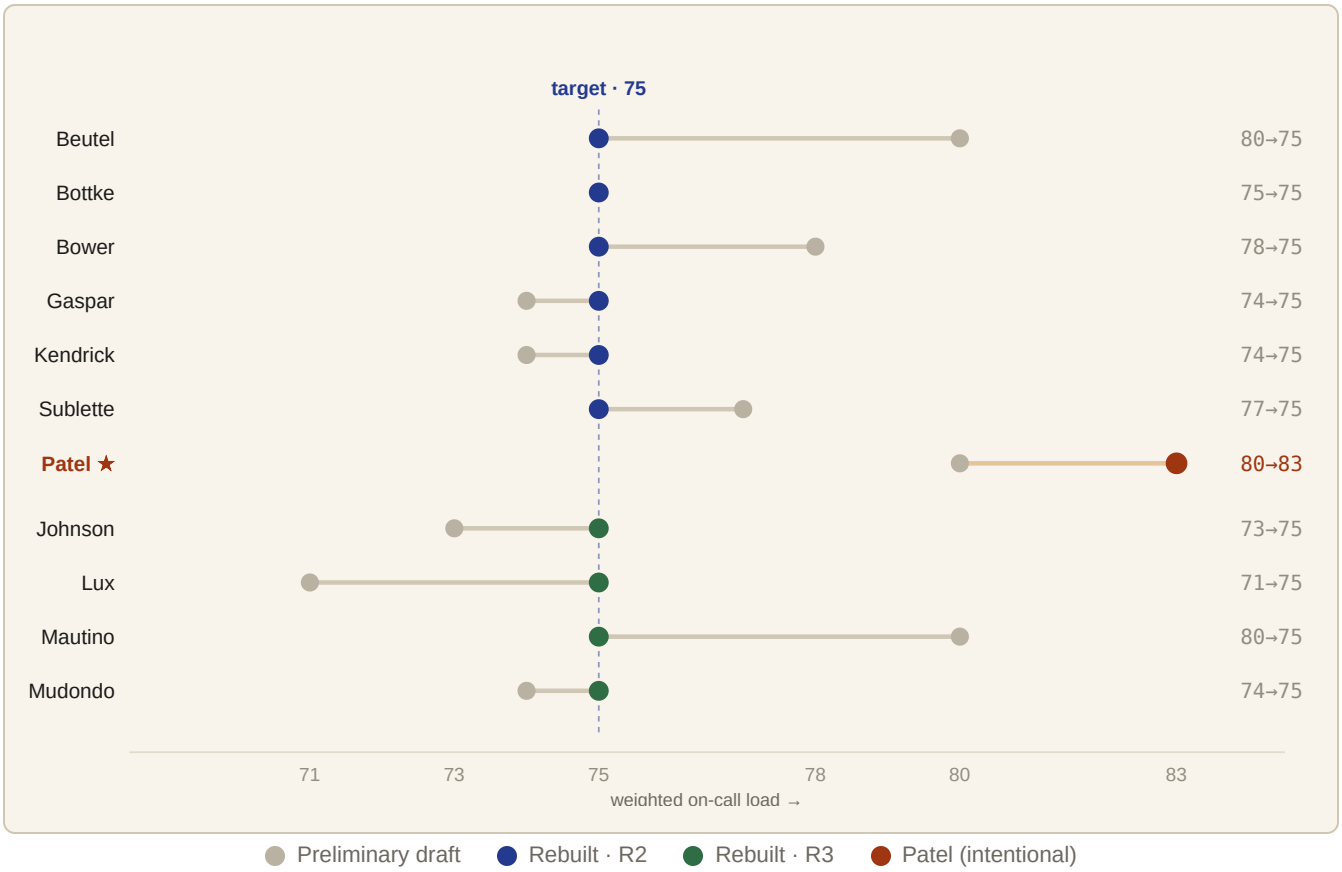
on a no-Sunday rotation — were relocated. The solver chose two replacements from **41** eligible candidate weekdays, leaving Patel as the schedule's single, intentional outlier and everyone else level beneath him.

FASCINATING FACT № 4 — THE PIGEONHOLE BEHIND "NO SUNDAYS"

There are 50 Sundays in the call year and, by rule, only seven R2s may cover them. Seven residents sharing 50 Sundays forces an average of 7.1 apiece — which is exactly why the structural rule lands at "7–8 Sundays each." The constraint is not arbitrary; it is the pigeonhole principle wearing a policy hat.

§ 8 The payoff, measured

The clearest way to see the optimization is to watch the senior weighted loads collapse from a ragged line onto a single value. Each resident below is a row; the grey dot is where the draft left them, the colored dot where the rebuild placed them.



Convergence to 75. The overloaded come down (Beutel, Mautino, Bower from 80–78); the underloaded come up (Lux from 71). The standard deviation of senior load falls from **3.0 to 0** across the ten balanced seniors — Patel alone is held high, on purpose.

3.0 → 0
STD. DEV. OF SENIOR WEIGHTED LOAD (10 BALANCED SENIORS)

4.0% → 0%
COEFFICIENT OF VARIATION, REGULAR SENIORS

0
ROTATION, PTO & BACK-TO-BACK CONFLICTS REMAINING

§ 9 Trust, but verify — all the way into the spreadsheet

A schedule no one trusts is no schedule at all. So the optimized calendar was written back into the original workbook format, and its live tally formulas were not replaced but *recomputed and checked*.

The workbook's per-month totals, day-of-week tables, and year-to-date rollups are driven by **1,430** spreadsheet formulas. Each one's value was recomputed independently and verified cell-by-cell against the solved schedule — all seventeen residents' totals and every day-of-week group matching, on every tab — while the formulas themselves were left intact so the workbook stays live. The final calendar passes the same machine audit that flagged the draft: zero conflicts, end to end.

The whole effort, by the numbers

10^{644}	Unconstrained candidate schedules — 645 digits, dwarfing the 10^{80} atoms in the universe	**354**	Call days assigned, each one intern + one senior
6,363	Variables in the model (6,059 of them boolean on/off switches)	**8,400**	Constraints across seven rule families
1,143	Resident-day pairs ruled out by rotation eligibility alone	**120**	Night-float weeknights pre-fixed by the rotation grid
8	Parallel search workers · 240-second solve budget · proven optimal on the core model	**75**	The weighted load shared, to the unit, by ten of eleven seniors
1,430	Workbook formulas recomputed and verified, cell by cell	**0**	Rule, PTO, and back-to-back conflicts in the final calendar

Methods & tools. Workbooks parsed and rewritten with Python and **openpyxl**; the schedule modeled as a constraint-optimization problem and solved with **Google OR-Tools CP-SAT** (lazy-clause-generation constraint programming). Fairness modeled as a min–max spread of day-of-week-weighted on-call load, balanced both within and across resident classes. Two deliverables accompany this note — **Version 1 (Minimal Repair)** and **Version 2 (Full Rebuild)** — each as a live workbook in the original tab format and as a standalone, interactive calendar. · UnityPoint Health Family Medicine Residency · Academic Year 2026–2027.